第11讲 文件管理&异常

马秀麟

2021.12

文件&文件管理

- 一、文件的概念
  - (1)什么是文件?
    - 文件的操作是系统中的最基本的数据处理形式,文件通常被存储在磁盘上，可以根据文件名来实施存取。
    - 文件及文件管理，是操作系统和任意一门编程语言必备的功能。
  - (2)文件的大类别
    - 在计算机中，从文档的可识别性来看，文件可以被分为两大类，一类是由纯粹文本（全部为可打印字符）构成的文件；另一类为二进制码构成的文档。
    - 对于纯文本文档，通常可直接打开并被用户阅读；而程序代码、图像、音频等都是二进制文档，这种文档不能直接被人们阅读，但可以通过软件读取并进行处理。

文件&文件管理

- 二、文件的操作
  - (1)打开文件
    - 含义
      - 文件的打开，可以是只读方式：采用r作为标记；也可以是写入方式：w。还可以加入“+”符号，表示对文档的内容进行添加或修改。
      - 文件的打开模式，可以是文本模式，采用t作为标记，也可以是二进制模式，采用b模式。
    - 命令执行格式：
      - 例如：
        - ff=open("文件名","rb")、
        - ff=open("文件名","wb+")等。

文件&文件管理

- 二、文件的操作
  - (2)文件读取
    - 含义
      - 从已经打开的文件里，读取所需的内容。
      - 要求打开方式为rb或rt
      - fo=open(“文件名”，“rb”)
    - 文件读取的模式：
      - 按行获取文件内容
        - for line in fo:
        - print(line)
      - 直接获取文档的全部内容
        - data=fo.read()

文件&文件管理

- 二、文件的操作
  - (2)文件读取
    - 针对巨大型文档的读取——每次读有限数据

```
while (True):
    data=fo.read(1024)    #最大不要超过16KB
    if not data:
        break
    else:
        pass              #处理语句
```

```
data=fo.read(1024)      #每次读1024个字节
while(True):
    若干数据处理语句
    data=fo.read(1024)
    if not data:          #若data为空，无数据
        break             #则结束循环
```

```
f.write("HelloWorld")

data = g.read()                 # Read all data

line = g.readline()             # Read a singleline

lines = g.readlines()           # Read data as a list of lines
```

文件&文件管理

- **二、文件的操作**
  - **(3)文件存储**
    - 含义
      - 向已经打开的文件里，写入已有的内容。
      - 要求打开方式为wb、或wt、或wb+、wt+等
      - fw=open(“文件名”，“wb+”)
    - 文件写入的模式：
      - 按行写入文件内容
        - fw.write(line)
      - 把缓存的内容写入文件内容
        - data=……..
        - fw.write(data)

11.2 Python读写文本文件

- 文件写入

```
f = open("d:\\file.txt","w")
f.write("HelloWorld")
f.close()
```

- 文件读取

```
f = open("d:\\file.txt","r")
content = f.read()
print(content)
f.close()
```

- 格式化的输入输出，使用%來格式化字串

文件&文件管理

- 二、文件的操作
  - (4)文件操作的应用
    - 要求（一）
      - 打开一个数据文件；
      - 每次读取1024字节
      - 循环使用密码做异或运算，实现文件的加密。
      - 把加密后的密文写入文档，得到了加密后的文档
    - 要求（二）
      - 按照前述思路对文档解密

With as

- 对于系统资源如文件、数据库连接、socket 而言，应用程序打开这些资源并执行完业务逻辑之后，必须做的一件事就是要关闭（释放）该资源。
- 比如 Python 程序打开一个文件，往文件中写内容，写完之后，就要关闭该文件，如果不关闭会出现什么情况呢？极端情况下会出现 Too many open files 的错误，因为系统允许你打开的最大文件数量是有限的。同样，对于数据库，如果连接数过多而没有及时关闭的话，就可能会出现 Can not connect to MySQL server Too many connections，因为数据库连接是一种非常昂贵的资源，不可能无限制的被创建。
- try except finally 可以解决上述问题，但这样的结构一多会导致整体代码结构 很臃肿繁琐，不易读、不美观。所以用with…as…语句对其进行简化。
- **with expression [as variable]:**
  with-block

https://blog.csdn.net/bradyM/article/details/125482733

```
with open(file, "w") as f:
        f.write("hello python")
```

- open 方法的返回值赋值给变量 f，
- 当离开 with 代码块的时候，系统会自动调用 f.close() 方法

- 例7-1 向文本文件中写入内容，然后再读出。

```
s = 'Hello world\n文本文件的读取方法\n文本文件的写入方法\n'

with open('sample.txt', 'w') as fp:     #默认使用cp936编码
    fp.write(s)

with open('sample.txt') as fp:          #默认使用cp936编码
    print(fp.read())
```

捕捉&处理异常

- **三、捕捉&处理异常的操作**
  - **(1)捕捉异常的概念及必要性**
    - 异常的概念
      - 对于Python程序的运行，出现错误有几种情况：(1) 程序中存在语法错误；(2)程序中存在着逻辑错误；(3)由于环境或文件的原因，导致了各种异常。
      - 对于前两种故障，是必须提前在编程时解决的——通过消除语法错误、排除逻辑不通，即可排除掉这些故障，使程序得以顺畅运行。
      - 而因环境问题或文件打开失败等原因而导致的故障，通常不是当前程序的问题，也无法预测——比如，因网络断开而导致获取网站数据失败，因远程数据库故障而导致访问数据库失败。——这种问题就是“异常”。
    - 捕捉异常的必要性
      - 为把程序的运行放置于可控范围，对涉及外部操作的语句，应该捕捉异常。
      - 提高程序的健壮性和容错性
      - 把晦涩难懂的错误提示转换为友好提示显示给最终用户

捕捉&处理异常

- **三、捕捉&处理异常的操作**
  - **(2)捕捉异常的方法**
    - 捕捉异常的语句

```
try:
    <有风险的语句块>（网络读取、文件操作）
except <异常类型>:
    <处理异常的语句块>
```

    - 或者：

```
try:
    <有风险的语句块（网络读取、文件操作）>
except:
    <处理异常的语句块>
```

捕捉&处理异常

- 三、捕捉&处理异常的操作
  - (3)捕捉异常的实用程序

```
try:
    fh = open('testfile', 'w')
    fh.write("这是一个测试文件，用于测试异常！！")
except IOError:
    print("Error: 没有找到文件或读取文件失败")
else:
    print("内容写入文件成功")
    fh.close()
```

```
try:
    x=input('请输入被除数: ')
    y=input('请输入除数: ')
    z=float(x) / float(y)
except ZeroDivisionError:
    print('除数不能为零')
except TypeError:
    print('被除数和除数应为数值类型')
except NameError:
    print('变量不存在')
else:
    print(x, '/', y, '=', z)
```

- 不建议使用异常来代替常规的检查，如if…else判断。
- 避免过多使用异常处理机制，只有再确实需要时才使用。
- 捕捉异常时，应尽量精准，并针对不同类型的异常设计不同的处理代码。

- Try……except……finally结构
- Finally子句中的内容无论是否发生异常都会执行，常用来做一些清理工作以释放try子句中申请的资源。
- Try:
- ……
- Finally:
- ……　#无论如何都会执行

- 使用异常处理结构保证文件总是能关闭

```
try:
    f = open('test.txt', 'r')
    line = f.readline()
    print(line)
finally:
    f.close()
```

■如果文件不存在，finally子句里的代码会抛出异常。

```
>>> try:
    f = open('test.txt', 'r')
    line = f.readline()
    print(line)
finally:
    f.close()

Traceback (most recent call last):
  File "<pyshell#17>", line 6, in <module>
    f.close()
NameError: name 'f' is not defined
```

- 如果try子句中的异常没有被处理，或者在except子句或else子句中出现了异常，那么这些异常将会在finally子句执行完后再次抛出。

```
>>> try:
    3/0
finally:
    print(5)

5
Traceback (most recent call last):
  File "<pyshell#52>", line 1, in <module>
    try:3/0
finally:
   print(5)
ZeroDivisionError: division by zero
```



- Python异常处理结构中可以同时包含多个except子句、else子句和finally子句，但一般不建议这么做。

```
>>> def div(x, y):
    try:
        print(x / y)
    except ZeroDivisionError:
        print('ZeroDivisionError')
    except TypeError:
        print('TypeError')
    else:
        print('No Error')
    finally:
        print("executing finally clause")
```



- 使用带有finally子句的异常处理结构时，应尽量避免在finally子句中使用return语句，否则可能会出现出乎意料的错误。

```
>>> def demo_div(a, b):
    try:
        return a/b
    except:
        pass
    finally:
        return -1
>>> demo_div(1, 0)
-1
>>> demo_div(1, 2)
-1
```



格式化类型文档的处理

- 四、针对格式化类型文档的处理
  - (1)打开图像文档
    - 采用基于pillow的Image对象
  - (2)打开数据文档
    - 打开Excel文档
      - 基于xlrd打开
        - 获取二维表格型数据
      - 基于Pandas打开
        - 获取DataFrame型数据
  - (3)打开音频文件
    - 基于pyAudio对象的打开

THANK YOU !